NOVIQ / FREE LEARNING LIBRARY

JavaScript Essentials Quick Reference

A practical beginner's guide to the language that makes the web interactive.

JS / 01 • PRACTICAL GUIDE • BEGINNER FRIENDLY

Clear explanations. Useful examples. Concepts you can apply.

01 / THE ROLE OF JAVASCRIPT

JavaScript adds behaviour to web pages. HTML describes the content, CSS controls presentation, and JavaScript responds to events, works with data and updates what a user sees. It can run in browsers and in server-side environments such as Node.js.

A useful mental model

Input → **logic** → **output.** A user clicks a button (input), your code performs a calculation or checks a condition (logic), and the page displays a result (output).

02 / VALUES, VARIABLES AND TYPES

Variables give names to values. Prefer **const** when you do not reassign a variable, and use **let** when the value needs to change. Avoid **var** in most modern beginner code because its scoping rules can be surprising.

```
const learner = "Asha";
let completedLessons = 2;
completedLessons = completedLessons + 1;
```

Common types include strings (text), numbers, booleans (true/false), undefined (a declared variable without a value), null (an intentional empty value), arrays (ordered lists) and objects (collections of named properties).

```
const course = { name: "Web Basics", lessons: 8, active: true };
const topics = ["HTML", "CSS", "JavaScript"];
```

03 / OPERATORS AND COMPARISONS

Arithmetic: +, -, *, /, % (remainder), ** (power). **Comparison:** ===, !==, >, <, >=, <=. **Logical:** && (and), || (or), ! (not). Use strict equality (===) in normal code so JavaScript does not silently convert unlike types.

```
const hasAccess = isLoggedIn && hasPaidPlan;
const isAdult = age >= 18;
```

04 / CONDITIONS AND LOOPS

Conditions let a program choose a path. Loops repeat work while a condition is true or for each item in a collection.

```
if (score >= 50) {
  console.log("Pass");
} else {
  console.log("Keep practising");
}

for (let i = 0; i < 3; i++) {
  console.log(i);
}
```

05 / FUNCTIONS

A function packages reusable logic. Parameters are the inputs; return sends a result back to the code that called the function. Give functions clear names and keep each function focused on one job.

```
function addTax(price, rate) {
  return price + price * rate;
}
const total = addTax(100, 0.18);
```

An arrow function is a shorter function expression:

```
const double = (number) => number * 2;
```

06 / ARRAYS AND OBJECTS

Arrays store ordered values. Array indexes start at zero. Objects represent one thing through named properties. These structures are used constantly when working with API responses and page data.

```
const tasks = ["Plan", "Build", "Test"];
tasks.push("Publish");
console.log(tasks[0]);

const user = { name: "Asha", role: "Designer" };
console.log(user.name);
```

Useful array methods

map() transforms each item; **filter()** keeps matching items; **find()** returns the first match; **forEach()** runs a function for each item. These methods do not all return the same kind of result—check the method's purpose before using it.

07 / THE DOM AND EVENTS

The Document Object Model (DOM) is the browser's structured representation of a page. JavaScript can select elements, change text and respond to events such as clicks or form submissions.

```
const button = document.querySelector("#save");
button.addEventListener("click", () => {
  document.querySelector("#status").textContent = "Saved";
});
```

Prefer textContent for inserting plain text. Be careful when inserting HTML from untrusted sources, because unsafe HTML can create security problems.

08 / ASYNCHRONOUS JAVASCRIPT

Some work takes time: requesting data, reading a file or waiting for a timer. Promises represent work that may finish later. async/await makes promise-based code easier to read, but errors still need handling.

```
async function loadPosts() {
  try {
    const response = await fetch("/api/posts");
    if (!response.ok) throw new Error("Request failed");
    const posts = await response.json();
    console.log(posts);
  } catch (error) {
    console.error(error);
  }
}
```

09 / DEBUGGING CHECKLIST

When something does not work, read the first useful error in the browser console. Check spelling and capitalisation, confirm that selected elements exist, inspect values with console.log, and reduce the problem to the smallest reproducible example. A syntax error can stop a script before later code runs.

10 / PRACTICE CHALLENGES

1. Create a function that converts minutes into seconds.
2. Given an array of prices, use filter() to keep prices above 500.
3. Make a button update a message on the page when clicked.
4. Fetch data from a public test API and show a loading message, the result or an error.

11 / QUICK REFERENCE

Task	Use
Declare a value	const / let
Compare strictly	=== / !==
Repeat over an array	for...of / forEach()
Transform an array	map()
Keep matching items	filter()
Select a page element	document.querySelector()
Listen for interaction	addEventListener()
Request web data	fetch()

Remember: practise by changing examples, predicting the output and explaining why the code behaves as it does. This reference is a companion to practice, not a substitute for testing your own code.

NOVIQ • Learn clearly. Build confidently.